

Session 1: AI for Coding – Tools, Concepts, and Setup

Elira Kuka & Tanner Regan

The George Washington University

May 4, 2026

What We Will Cover (Sessions 1 & 2)

- ▶ Part 1: Tools, concepts, and setup
 - Overview of AI coding tools and how to choose among them
 - How to use AI Agents: Terminal, Desktop App, VS Code
 - Conceptual overview of context engineering with Claude
 - VS Code and Claude Code integration
 - Git and Github: why they matter for research and Claude

What We Will Cover (Sessions 1 & 2)

- ▶ Part 1: Tools, concepts, and setup
 - Overview of AI coding tools and how to choose among them
 - How to use AI Agents: Terminal, Desktop App, VS Code
 - Conceptual overview of context engineering with Claude
 - VS Code and Claude Code integration
 - Git and Github: why they matter for research and Claude
- ▶ Part 2: Practical workflows
 - Writing, debugging, and automating code with AI
 - “Vibe coding” from plain English descriptions
 - Project organization and replication packages
 - Demo: data cleaning script from scratch (python and stata)

Coding Perhaps Highest-Value Use of AI

- ▶ AI most reliable when output is verifiable: code runs or it does not
- ▶ Economists spend enormous time on repetitive coding tasks
 - AI can handle most of them
- ▶ Massive productivity gains across skill levels:
 - Beginners: “vibe code” to write code in an unfamiliar language (no syntax!)
 - Advanced users: automate drudge work, prototype faster, reduce bugs

Coding Perhaps Highest-Value Use of AI

- ▶ AI most reliable when output is verifiable: code runs or it does not
 - ▶ Economists spend enormous time on repetitive coding tasks
 - AI can handle most of them
 - ▶ Massive productivity gains across skill levels:
 - Beginners: “vibe code” to write code in an unfamiliar language (no syntax!)
 - Advanced users: automate drudge work, prototype faster, reduce bugs
- ⇒ First two sessions will cover how to use AI to complete coding tasks

Overview of AI Coding Tools

The AI Coding Landscape: Many Options

- ▶ General-purpose AI assistants (good at coding, among many other things):
 - **Claude** (Anthropic) — our focus today
 - **ChatGPT / GPT-4o** (OpenAI)
 - **Gemini** (Google)
- ▶ Agentic coding tools (read files, write and run code autonomously):
 - **Claude Code** (Anthropic) — our focus today
 - **Codex** (OpenAI) — closest competitor to Claude Code
 - **GitHub Copilot** (Microsoft) — inline suggestions in editor
 - **Cursor / Windsurf** — AI-first code editors
- ▶ How to choose: task type, privacy needs, and workflow integration

Chatbot vs. Agentic Tool

- ▶ **Chatbot** (Claude.ai, ChatGPT): you ask → it answers → *you* act
- ▶ **Agentic tool** (Claude Code): you give a goal → it reads files, writes code, runs it, fixes errors, iterates → reports back
- ▶ For economists: instead of “here is some code,” it runs the code and tells you what it found

Which One Should You Use? Claude Code vs. Codex

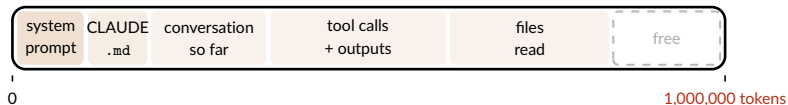
- ▶ Two serious options for agentic coding today: **Claude Code** and **Codex**
- ▶ Plans mirror each other at each price point:
 - \$20/mo: Claude Pro vs. ChatGPT Plus
 - \$100/mo: Claude Max (5x) vs. ChatGPT Pro (5x)
 - \$200/mo: Claude Max (20x) vs. ChatGPT Pro (20x)
- ▶ Both use a 5-hour rolling window + weekly cap; limits reset automatically
- ▶ At the same price, Codex currently gives more usage per dollar

Which One Should You Use? Claude Code vs. Codex

- ▶ Two serious options for agentic coding today: **Claude Code** and **Codex**
- ▶ **Codex** has some advantages:
 - More usage per subscription dollar — important for heavy users
 - Sometimes stronger on pure coding benchmarks
- ▶ **Claude Code** has some advantages:
 - Often better at tasks beyond code: understanding context, writing, reasoning
- ▶ **Bottom line:** this comparison is fast-moving and will keep changing
 - I am using Claude Code and find it excellent for my workflow
 - I would switch only if Codex proved clearly and consistently superior
 - My advice: pick one, learn it well, reassess in 6 months

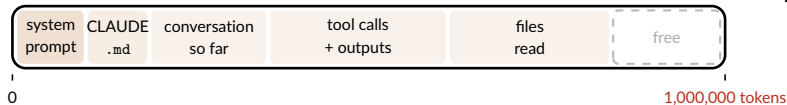
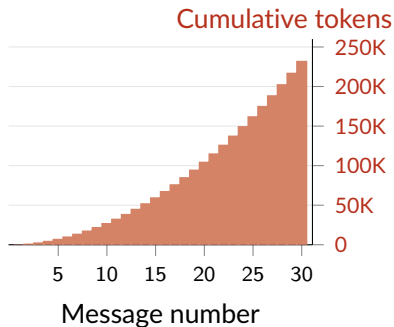
What You Pay For: Tokens and Context

- ▶ **Token:** the unit AI reads and writes — roughly $\frac{3}{4}$ of a word
 - **You pay for:** tokens *in* (prompt + files) and tokens *out* (reply) — input is cheaper
- ▶ **Context window:** everything the model holds at once — fills as session grows



What You Pay For: Tokens and Context

- ▶ **Token:** the unit AI reads and writes — roughly $\frac{3}{4}$ of a word
 - **You pay for:** tokens *in* (prompt + files) and tokens *out* (reply) — input is cheaper
- ▶ **Context window:** everything the model holds at once — fills as session grows



Getting the Most Out of Your Subscription

- ▶ Subscriptions have rolling limits — fewer tokens per task means more use
- ▶ How to use fewer tokens:
 - Start a new session for each new task: context accumulates and raises costs
 - Store standing instructions once so you never repeat yourself
 - Be specific: vague prompts waste tokens and invite unfocused replies
 - Ask multiple things at once so less back and forth
 - Compact your chats/context window
- ▶ Silly trick: start Claude when wake up so 5-hour window ends earlier in day

Contex Engineering with Claude Overview

Context Engineering

▶ Context Window

- ▶ Each prompt to Claude Code bundles the entire history of your session
 - ▶ your prompts, Claude's responses, files it read, code outputs, system prompts
- ▶ These are all appended until the context window hits its fixed size limit
 - ▶ For Claude Code this is around 200k tokens
- ▶ Claude then “**auto-compacts**” writes its own summary and then restarts fresh
 - ▶ less preferable to frequent intentional compaction

▶ Context Engineering

- ▶ You can tell Claude when to compact using `/compact your instructions`
- ▶ Develop habits that the context window focussed on relevant information
 - ▶ Claude.md vs skills —and— ‘ask before edits’ vs ‘plan’ Modes

Read more at [Paul GP's Guide](#)

Anatomy of a context window (example from Paul GP's Guide)

```
<|start|>system<|message|>You are ChatGPT, a large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-06-28  
Reasoning: high  
# Valid channels: analysis, commentary, final.  
Channel must be included for every message.  
Calls to these tools must go to the commentary channel: 'functions'.<|end|>
```

system prompt
(written by openAI)

ANATOMY OF A
MODERN LLM
CHAT

```
<|start|>developer<|message|># Instructions  
Be concise and technical. Provide code examples when relevant.
```

developer
message
(tool definitions)

```
# Tools  
## functions  
namespace functions {  
  type web_search = (_: {  
    query: string,  
    num_results?: number,  
  }) => any;  
} // namespace functions<|end|>
```

user
prompt

```
<|start|>user<|message|>What's new in the latest version of polars?<|end|>
```

thinking

```
<|start|>assistant<|channel|>analysis<|message|>User asking about latest polars version.  
My knowledge cutoff is 2024-06, need to search for recent updates.<|end|>
```

```
<|start|>assistant<|channel|>commentary<|message|><|call|>functions.web_search  
{  
  "query": "polars python latest version new features", "  
  "num_results": 5  
}<|end|>
```

tool call

```
<|start|>tool<|message|>[{"title": "Polars 1.8.0 Release Notes", "snippet": "New  
streaming engine, improved join performance, added pivot_wider...", "url": "..."}]<|end|>
```

tool
results

```
<|start|>assistant<|channel|>final<|message|>Polars 1.8.0 (latest as of Nov 2025)  
includes:
```

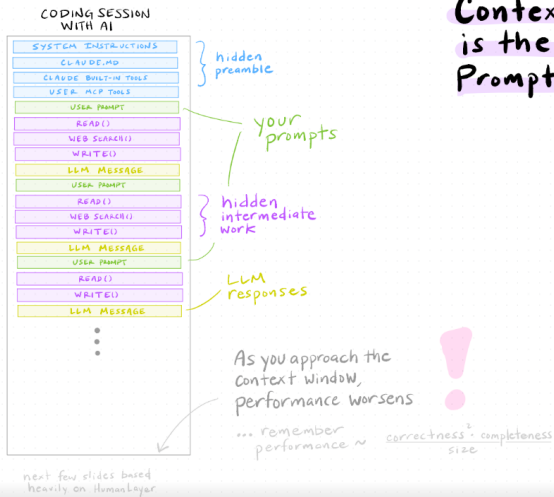
final response

```
- **Streaming engine overhaul**: Better memory efficiency for large datasets  
- **Join performance**: 2-3x faster for certain join types  
- **New API**: `pivot_wider()` for reshaping data  
- **LazyFrame improvements**: Enhanced query optimization
```

```
Check the release notes for migration details if upgrading from older versions.<|return|>
```

You
typically
see only
these
two!

Context engineering (example from Paul GP's Guide)



Context Engineering
is the new
Prompt Engineering

Context auto-compact (example from Paul GP's Guide)

If you hit the content window the LLM will auto-compact



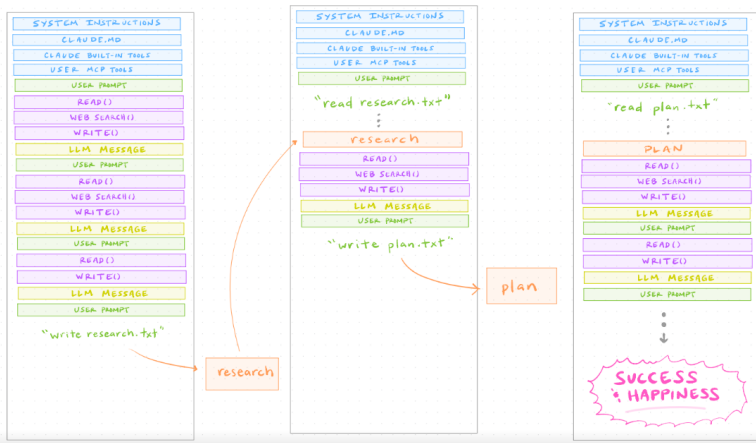
the model summarizes your conversation

...this is rarely ideal...



Context intentional compacting (example from Paul GP's Guide)

Frequent intentional compaction is better



CLAUDE.md and Skills

- ▶ **CLAUDE.md** (memory) — Claude reads at the start of every session
 - Store context that is always true so you don't repeat yourself in every prompt
 - `.claude/CLAUDE.md` is global, `your-project/CLAUDE.md` is *project* specific
 - Ask Claude to update after substantial changes to project
- ▶ **Skills** — Claude reads every time it is called
 - Store context for a task that you repeat often, but not always relevant
 - `.claude/skills/your-skill/SKILL.md` is called with `/your-skill`
 - Write and update your own skills or download from others

Read more at [Claes Backman's Guide](#) (full descriptions, examples of CLAUDE.md and skills, download pre-made skills)

your-project/CLAUDE.md (example from Claes Backman's Guide)

```
# Project: [Paper title]
```

Data

- Unit of analysis: firm-year panel, 2010-2022
- Raw data lives in data/raw/ – never edit these files
- Main dataset after cleaning: data/clean/panel.dta (N $\approx$ 47,000)

Code conventions

- R scripts are numbered (01_clean.R, 02_analysis.R, ...)
- All regressions cluster SEs at the firm level
- Use fixest for all panel regressions

LaTeX

- Main file: paper/main.tex
- Tables generated by 03_tables.R go to paper/tables/
- Use `\widehat{}` not `\hat{}` for estimators

`.claude/skills/robustness/SKILL.md` (example from Claes Backman's Guide)

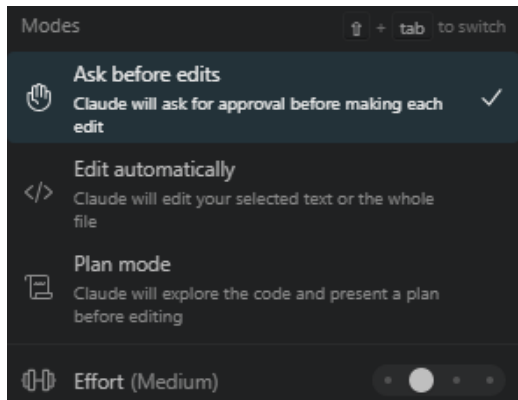
```
---  
name: robustness  
description: Run standard robustness checks on the main regression.  
---
```

For the main specification in this project:

1. Re-run with alternative clustering (industry-year instead of firm)
2. Re-run dropping the top and bottom 1% of the outcome variable
3. Re-run on the pre-2020 subsample only
4. Produce a summary table comparing coefficients across all specs

Claude Modes

- ▶ **Ask before edits**
 - ▶ Do steps one at a time
 - ▶ Asks your permission before running each and every command
- ▶ **Edit automatically**
 - ▶ Same as above but runs commands without asking permission (risky)
- ▶ **Plan**
 - ▶ Claude writes plan.md with many steps
 - ▶ You review and execute entire plan



Other tips for managing the context window

- ▶ **Use @-mentions to be selective**

- ▶ Rather than Claude reading everything, point to a specific file

- ▶ e.g. `update all the pathnames in @Lecture_2.tex`

- ▶ **Check usage with /context**

- ▶ Type `/context` in the panel to see a breakdown of how your context window is being used and how much space remains.

Files and Data that Claude is good at reading

- ▶ Claude Code works best with plain text files
 - ▶ e.g. `.txt`, `.do`, `.py`, `.tex`, `.csv`, `.log`, `.md`, etc.
- ▶ Tips for optimal performance with other file types
 - ▶ `.docx`: convert to markdown with `pandoc file.docx -o file.md`
 - ▶ `.xlsx`, `.dta`, `.rds`, `.sav`: convert to CSV first with a short script
 - ▶ `.pdf`:
 - ▶ Use the `.tex` file whenever possible (especially if equations and tables)
 - ▶ Long PDFs (50+ pages): convert to markdown first
 - ▶ Scanned PDFs: run OCR first, then convert to markdown
- ▶ Create a skill that loads file types, e.g. `/pdf-to-markdown`

Read more at [Claes Backman's Guide](#) (details on handling different file types)

Claude Code / Codex Interfaces

Three Ways to Use an Agentic Coding Tool

▶ **Terminal** (command line):

- Most powerful; works directly in your project directory
- Best for: running scripts, data analysis, replication tasks

▶ **Desktop App:**

- Friendlier interface; easier to get started
- Projects feature: persistent context across sessions
- Best for: drafting, quick questions, early exploration

▶ **VS Code integration:**

- Agent lives inside your editor — see code and AI side by side
- Best for: sustained development work, Git integration, Jupyter/Python

Terminal vs. VS Code: Quick Comparison

Terminal (CLI)

- ▶ Lightweight; fast startup
- ▶ Works anywhere, incl. remote servers
- ▶ Keyboard-driven; fast for quick tasks
- ▶ No inline diffs; steeper setup

VS Code Extension

- ▶ Inline diffs; accept changes block-by-block
- ▶ Code & AI panel side by side
- ▶ Best for large codebases, sustained work
- ▶ Heavier on CPU and memory

Live Demo: Terminal

Live Demo: Desktop App

Live Demo: VS Code

VS Code with Claude Overview

What is VS Code?

- ▶ Integrated Development Environment (IDE)
 - ▶ Free, open source (created by Microsoft), and very popular
- ▶ Core features:
 - **Explorer:** file browser and project tree
 - **Editor:** syntax highlighting for every language (Stata, R, Python, $\text{\LaTeX}$, ...)
 - **Integrated terminal:** run scripts without leaving the editor
 - **Extensions:** Git, Claude, Stata, latex, python, etc. [Starter Pack for Economists](#).
 - **Command Palette** (`Ctrl+Shift+P`): search and run any action by name
- ▶ Claes Backman's Guide [Claude Code in VS Code for Academic Economists](#)
 - ▶ Other resources: [Official intro videos \(Microsoft\)](#), [Tips and Tricks \(Microsoft\)](#)

GWU_AI_study_group > slides > Lecture_1.pdf

27/40

What is VS Code?

- ▶ **Integrated Development Environment (IDE)**
 - ▶ Free, open source (created by Microsoft), and very popular
- ▶ **Core features:**
 - **Explorer:** file browser and project tree
 - **Editor:** syntax highlighting for every language (Stata, R, Python, ~~Excel~~, ...)
 - **Integrated terminal:** run scripts without leaving the editor
 - **Extensions:** Git, Jupyter, Claude Code, and more
 - **Command Palette** (`Ctrl+Shift+P`): search and run any action by name
- ▶ **Resources:** [Official intro videos \(Microsoft\)](#), [Tips and Tricks \(Microsoft\)](#)

38/40

The screenshot displays the Visual Studio Code (VS Code) integrated development environment. The main editor window shows a Python script with syntax highlighting. The left sidebar contains the Explorer view (showing a file tree) and the Search view. The right sidebar shows the Extensions view. A central panel, likely a web browser or a dedicated overview window, displays the 'VS Code with Claude Overview' page, which provides information about the VS Code ecosystem, including its features, extensions, and how to get started. The bottom status bar indicates the current file is a Python script and that the Jupyter extension is active.

VS Code with Claude Overview

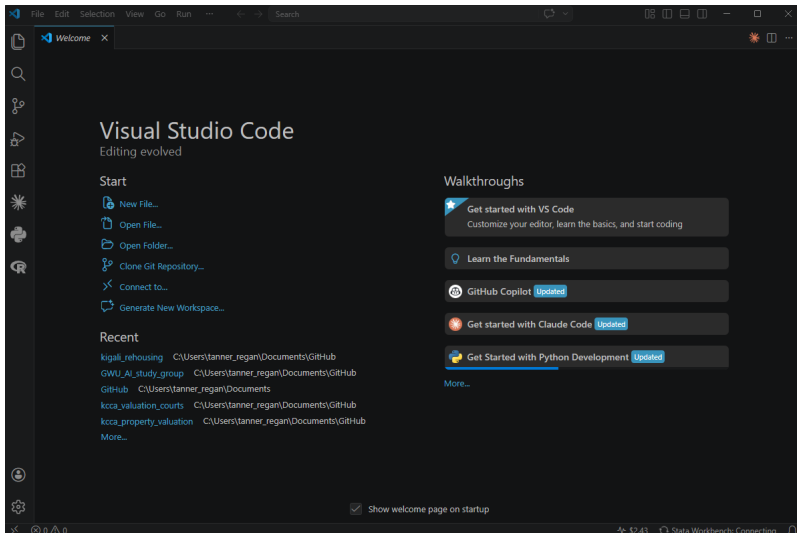
- The slide now uses Beamer's `columns` environment — list on the left half, image on the right half, both taking 48% of the text width with a small gap between them.

 **Ask before edits**

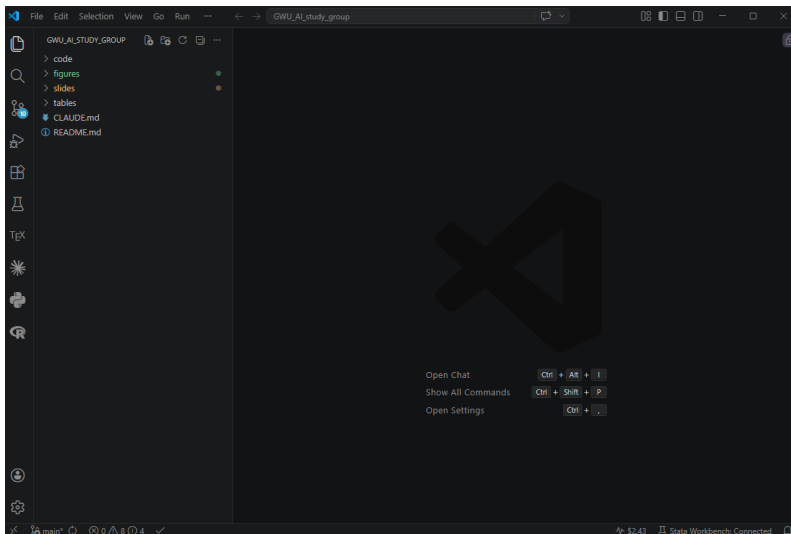
Claude Code **extension** in VS Code

- ▶ Give Claude access to your whole project directory to maximize its usefulness
 - ▶ Open your full project folder in VS Code (File → Open Folder)
 - ▶ This can be any folder on your computer — plain folder, a git repository, a Dropbox folder, etc.

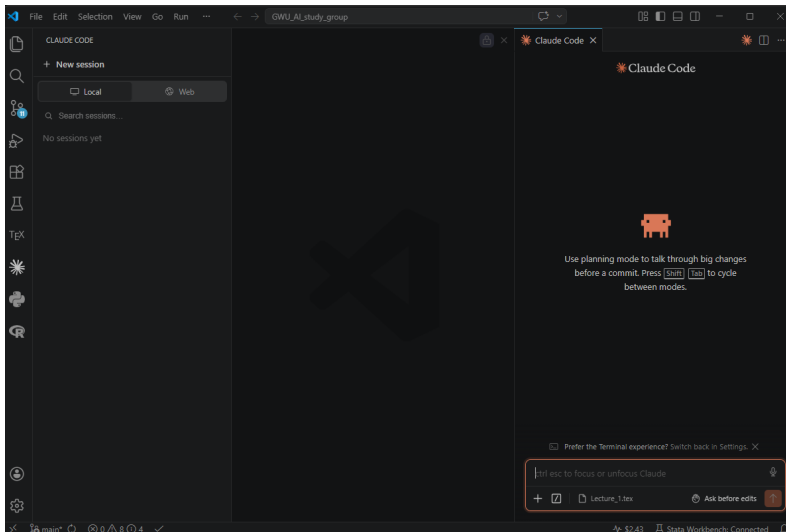
New VS Code Window — open new folder



Project folder is open — now open a new Claude Code session

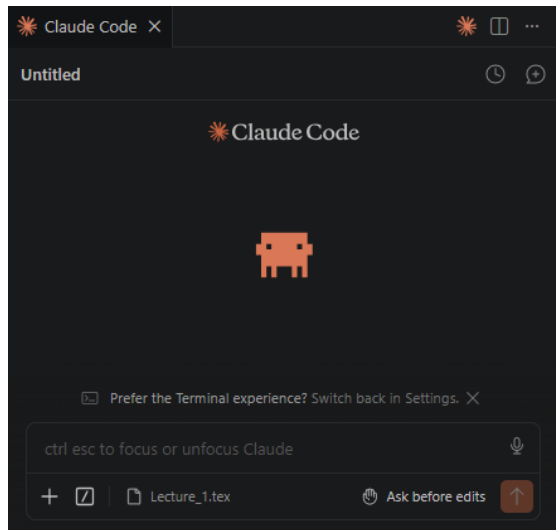


New Claude Code session is open



Claude Code extension in VS Code

- ▶ Each new conversation in the Claude panel is a fresh context
 - ▶ Click the + button at the top of the Claude panel to start one,
 - ▶ or type `/clear` to reset
- ▶ Go over a fresh window →
 1. Session Title
 2. Session History
 3. Mode menu
 4. Current File Selection
 5. Attachments
 6. Command Menu



Git: Version Control for Coding Projects

Why Git Matters for Research

- ▶ Version control = a complete history of every change to your code
 - ▶ No more: `analysis_v3_FINAL_revised2.do`
- ▶ Enables:
 - Backup: retrieve any prior version of your code
 - Collaborate: coauthors and RAs edit without overwriting each other
 - Track: what changed and who made the changes at every stage
- ▶ Can help improve project replicability both internally and externally
 - ▶ an automatic audit trail
 - ▶ sharing and iterating on projects publicly

Read more at Luís Fonseca's [Version Control with git \(for economists\)](#)

Git and Github Concepts

- ▶ Key concepts to remember
 - ▶ **Repository (repo):** a folder whose history Git is tracking
 - ▶ **Commit:** a snapshot of the code at a point in time, with a descriptive message
 - ▶ **Sync:** Pushes and pulls at the same time, so both repos are up to date
- ▶ Slightly more advanced concepts (learn these when you need them)
 - ▶ **Push:** sends changes up from your local repo to online/remote repo
 - ▶ **Pull:** brings changes down to your local repo from online/remote repo
 - ▶ **Branch:** a parallel version of the code (useful for exploration and RA work)
 - ▶ **Merge:** fold a branch back into the main code
 - ▶ **Pull request:** propose to merge one branch to another (e.g. main)
 - ▶ **Clone:** copy an online repo to local computer for initial setup
 - ▶ **Fork:** copy an online repo to your own GitHub account

Git vs. GitHub: Two Separate Things



git — the software that runs locally

- ▶ All history lives in the `.git` folder
- ▶ Records *commits*, *branches*, *merges*
- ▶ Compares any two versions line-by-line; flags conflicts



GitHub — online hosting platform

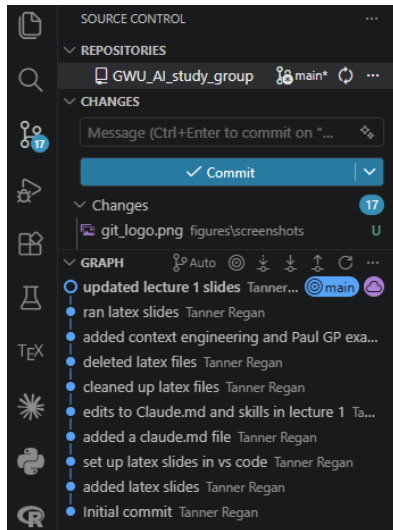
- ▶ Stores a remote copy of your repo
- ▶ Alternatives: GitLab, Bitbucket
- ▶ Remote endpoint for *Push*, *Pull*, *Clone*; introduces *Fork*

Typical Git workflows for an economist

- ▶ Typical workflow for an economist (alone or with trusted RA)
 1. Pull/Sync code at start of work session (get latest version of online repo)
 2. Work on code; when something works, *commit* it with a descriptive message
 3. Push/Sync at end of work session (updates the online repo)
- ▶ More elaborate, with explicit RA review
 1. RA syncs and then creates a new branch
 2. RA works on branch, makes commits and makes a pull request
 3. You review the pull request (compare the branch to main)
 - ▶ recall git highlights differences line by line and flags conflicts
 4. You merge back into *remote* main branch, or ask RA for fixes
 5. Pull/Sync to update your local repo

Git source control panel in VS Code

- ▶ Source Control panel
 - ▶ Click the github icon to open
- ▶ Go over an example →
 1. Icon with number of changes
 2. Message bar for commit
 3. Pull/Sync button
 4. commit button (also push)
 5. List of changes
 6. Graph of commits and branches



Setting up a github repository

- ▶ Go to github.com
 - ▶ Create an account, add new repository, add collaborators
 - ▶ Create a readme file that describes the project
- ▶ Open VS Code git extension (and sign into your GitHub account)
 - ▶ clone the repo to local computer: `Ctrl + Shift + P` (Windows/Linux) or `Cmd + Shift + P` (Mac) to open the Command Palette. Type `Git: Clone` and press Enter. Choose from the list of your repos.
- ▶ I have created a GitHub repo for our workshop [GWU_AI_study_group](#)
 - ▶ It is setup as Public, so you can already look at it and even Fork it
 - ▶ If you make a GitHub account and send me your username, I will add you as a collaborator so you can push changes to the repo as well

What Goes in a Repository?

- ▶ **Do NOT** store any datasets (source or generated)
 - instead store in **Box** (GW-endorsed secure storage)
 - or locally, Dropbox, Google Drive, etc.alternatively, write a `.gitignore` file that tells git to ignore specific subfolders (e.g. `/data`)
- ▶ **Do** store all other project materials
 - code (Stata, Python, $\text{\LaTeX}$)
 - output: tables, figures
 - CLAUDE.md, notes, slides

```
GWU_AI_study_group/  
├── slides/  
│   ├── _preamble.tex  
│   ├── Lecture_1.tex  
│   └── Lecture_2.tex  
├── code/  
├── figures/  
├── notes/  
├── tables/  
├── CLAUDE.md  
└── README.md
```

Git and Claude Code Integration

- ▶ Claude Code is get-aware
 - ▶ When the open project is a git repo, Claude can see the entire commit history
- ▶ Claude can facilitate git operations
 - ▶ Write commit messages `commit my changes with a descriptive message`
 - ▶ Explain what changed `what changed between commit A and B?`
 - ▶ Resolve a merge conflict `fix RA code to merge with the main branch`

Read more at [Claes Backman's Guide](#)

Setting Up Your Environment

Core tools needed to set yourself up

- ▶ Software to install (first two are substitutable)
 1. **VS Code** — free to download [here](#)
 - ▶ + Claude Code extension
 2. **Claude Code** — free to download [here](#)
 3. **Git** — free to download [here](#)
- ▶ Subscriptions and accounts to create
 1. **Claude Code** — subscribe for a fee ([here](#))
 2. **GitHub** — free account ([here](#)), and send Tanner your username!
- ▶ Guides
 - ▶ A step-by-step setup guide for Mac ([here](#)) and Windows ([here](#))
 - ▶ However, Claude can walk you through any setup as long as you:
 - ▶ install VS Code + Claude Code extension
 - ▶ or install Claude Code desktop app

Other set up if you want to follow along next class

- ▶ Software to install (first two are substitutable)
 1. **Miniconda** — free to download [here](#), python will come included
 2. **MikTeX** — free to download [here](#), I also had to install [Strawberry Perl](#)
 3. **Stata** — our old friend
- ▶ VS Code Extensions
 - ▶ Claude Code, LaTeX Workshop, Stata Workbench, Python, Jupyter
 - ▶ See also Claes Backman's [Starter Pack for Economists](#)

Session 1 — Key Takeaways

1. AI has perhaps most value added in coding — output is verifiable
2. **Agentic** $\neq$ **chatbot**: reads files, writes and runs code, fixes errors, reports back
3. Two serious options today: **Claude Code** and **Codex** — pick one
4. **Manage your context window**: tokens accumulate fast — start fresh sessions, compact often, use CLAUDE.md so you never repeat yourself
5. **Git + GitHub** are your safety net — commit before big changes, push often
6. Three interfaces (Terminal, Desktop, VS Code) — VS Code integrates everything

Questions?

- ▶ Questions on tools, concepts, or setup?
- ▶ Thoughts on how this fits your current workflow?

Clarifications from day 1:

- ▶ Context window $\neq$ agent.
 - ▶ Can ask the 'main' agent, to spawn **subagents** to answer a prompt. Distinct from **agent teams** where multiple claude sessions work together
- ▶ **Cowork vs Claude Code**
 - ▶ Cowork comes with the desktop app
 - ▶ It restricts which IP addresses the agent can reach (can adjust permissions)
 - ▶ It's less risky, but less powerful than CC due to the restrictions
- ▶ Permissions tips:
 - ▶ set **/.claude/settings.local.json** so certain domains are always allowed in project

Questions from workshop 1, consider exploring in later sessions:

1. How can we get 'stable' or 'replicable' results from Claude or LLMs in general?
 - ▶ Partly why CC is attractive - code is verifiable and generates replicable results
 - ▶ What is the state of the literature on this?
 - ▶ Can 'temperature' be set with Claude Code?
2. Compare Codex and Claude more closely (Paul GP uses both simultaneously)
3. Accessing a remote server and using Claude in same workflow.
 - ▶ Does Claude need to be installed on the server? I think no.
 - ▶ How to set up VS Code with a remote server
 - ▶ both Claes Backman and Paul GP mention this
 - ▶ Working with GW resources: [CAAREN](#), [High Performance Computing \(HPC\)](#), [Research Cloud](#), [Research Storage \(NAS\)](#), [Virtual Computer Lab \(VCL\)](#) , e.g. [Stata](#)